GenXTaxi — Project Documentation

GenXTaxi is a ride-hailing platform: a rider app, a driver app, an admin web panel, a Node.js backend, and a separate AI platform (chatbot, review summaries, demand/surge advisory).

#	Document	Read it for
1	Architecture & Repositories	How the pieces fit, tech stack, repos, data flow
2	Product & Feature Guide	What each app/panel does, ride lifecycle, business rules
3	Backend & API Reference	Modules, REST surface, sockets, data models, auth
4	Mobile Apps	Rider/driver app structure, screens, build & release
5	Admin Panel	Admin pages and what each manages
6	AI Platform	Gateway, chatbot, insights, demand services
7	Setup, Deployment & Operations	Run locally, env vars, Render, runbooks, security
8	Testing & Known Issues	What was tested, results, open bugs, gaps

Accuracy note: this was written from the code and from hands-on testing in October 2026. Items marked **(unverified)** were read from code but not exercised.

1. Architecture & Repositories

1.1 Repositories

Folder	Git remote	Stack	Role
`gen-taxi-backend/`	`arun1305/genXtaxi-backend` (main)	Node 20, Express, MongoDB/Mongoose, Redis, Socket.IO	Core API + realtime
`gen-taxi-frontend/`	`arun1305/genXtaxi-Mobile` (branch `ai-features`)	Expo SDK 50, React Native, TypeScript	One codebase → rider app and driver app (`APP_VARIANT`)
`gen-taxi-admin/`	`arun1305/genXtaxi-Admin` (main)	React, Vite, TypeScript, Tailwind, React Query	Admin dashboard
AI platform (repo root)	`arun1305/genXtaxi-ai` (main)	NestJS services + one FastAPI service, pnpm workspace	`ai-gateway`, `currency-service`, `chat-orchestrator`, `insights-service`, `demand-service`, `packages/ai-shared`, `ai-eval`
`genxtaxi-nestjs-backend/`, `gen-taxi-landing/`	—	NestJS/Prisma; landing site	Not part of the live flow described here **(unverified)**

1.2 System diagram

```
 Rider app ─┐                                          ┌─ MongoDB (genxtaxi)
 Driver app ─┼─ REST (JWT) + Socket.IO ─► Backend API ─┼─ Redis (geo, cache, pub/sub)
 Admin web ─┘                                          └─ Stripe/Razorpay, Twilio/MSG91, FCM, S3,
Google Maps
      │
      └─ AI calls ─► ai-gateway ─► Groq / Anthropic (LLMs), embeddings
                        ├─ chat-orchestrator  (support chatbot, tools call the Backend REST
API)
                        ├─ insights-service   (review summaries; reads rides collection)
                        ├─ demand-service     (FastAPI; demand forecast + advisory surge)
                        └─ currency-service   (FX)
```

All services share one JWT secret (`JWT_SECRET`) so a token issued by the backend is accepted everywhere. JWT payload: `{ id, role, sessionId, jti }`; roles are `passenger`, `driver`, `admin`.

1.3 Key design decisions

- **One mobile codebase, two apps.** `APP_VARIANT=driver|passenger` selects package ID, navigator and branding.
- **Backend dispatch is server-driven**: Redis geo index of online drivers, expanding search radius (3→5→8→12 km), one offer at a time, ~30 s offer window, retry loop.
- **Pickup OTP gate**: a trip cannot start until the driver enters the 4-digit OTP shown to the rider.

- **Surge**: the live backend has a polygon/time-window surge engine (`modules/pricing`). The AI `demand-service` surge is *advisory/shadow only* and does not change fares.
- **AI platform is separate** so it can scale and fail independently; the apps degrade gracefully when it is down.
- **Refunds always queue to an admin** (never auto-approved by the chatbot).

1.4 Environments

- **Production (Render, Singapore):** `gentaxi-backend` , plus five AI services (`gentaxi-ai-gateway` , `-currency` , `-chat` , `-insights` , `-demand`). Free/starter plans: services sleep when idle (cold start ≈ 30–50 s).
- **Local:** backend :5001, admin :5173, Mongo :27017, Redis :6379 (see doc 7).

2. Product & Feature Guide

2.1 Rider app

Authentication (phone + OTP, social login), profile registration, home map, destination/pickup search (Google Places), ride types with fare estimate and ETA, promo codes, payment method selection, ride request and live search, driver assignment (name, rating, vehicle, plate, pickup OTP), live tracking, share trip, SOS (hold 2 s), cancel with fee notice, trip completion with fare breakdown, rating + tags + review, ride history and receipts, wallet (balance, top-up, history), saved addresses, emergency contacts, notification preferences, help & support, terms/privacy, AI booking assistant, AI support chat, dark mode, pool/scheduled booking entry points.

2.2 Driver app

Phone/OTP login, onboarding and document upload, vehicle info (admin-managed), online/offline toggle, live ride offers (accept/decline with countdown), navigation to pickup, "arrived", OTP verification, start trip, in-trip navigation, complete trip with earnings breakdown, rate passenger, ride history, earnings (today/week/month/all with trip breakdown), wallet and withdrawals, scheduled availability, subscription plans (fixed fee vs commission), notifications, help & support, My Feedback (AI review insights), AI assistant.

2.3 Admin panel

Dashboard, analytics, live map (rides, drivers, SOS), dispatch (manual bookings), rides, users, drivers (approval/suspend/documents), vehicles, payouts, settings (provider integrations), audit log, ride types, document requirements, subscriptions, pricing, surge zones, promos/referrals, platform and branding pages, and AI pages (cost, currencies, knowledge base, handoff inbox, review insights, surge, AI ops). Details in doc 5.

2.4 Ride lifecycle

1. **Estimate** (`POST /rides/estimate`) → fare from distance/time/ride type/surge/promo.
2. **Request** (`POST /rides`) → status `searching` ; pickup OTP generated.
3. **Dispatch** → nearest eligible driver gets `ride:offer` ; decline/timeout → next driver/wider radius; none → `no_driver` .
4. **Accept** → `accepted` ; rider sees driver + OTP.
5. **Arrived** → `driver_arrived` ; driver verifies OTP (`/rides/:id/verify-pickup-otp`).
6. **Start** → `in_progress` .
7. **Complete** → `completed` ; fare finalised, driver earning + revenue ledger written, payment status `paid` (cash) or `pending` (card/wallet until settled).
8. **Rate** (both sides). Cancellation allowed before start (fee tiers: grace / en-route / after-arrived).

Allowed transitions: `accepted→driver_arrived|cancelled` , `driver_arrived→in_progress|cancelled` , `in_progress→completed` .

2.5 Money

Fare = base + distance + time (+ surge, night surcharge, booking fee, stops) − discount, plus tax; platform fee (default 20 %) and driver share (80 %) come from the pricing config, versioned per ride. Driver subscriptions replace commission with a fixed fee. The app UI renders amounts in **₹ (INR)**; backend defaults were aligned to INR in Oct 2026. Multi-currency display is **not** implemented in the apps (symbol is hard-coded).

2.6 Safety

SOS creates an alert (deduplicated per user), tracking link, SMS to emergency contacts, admin live-map alert; admins triage and resolve with notes; unresolved alerts escalate after 5 min.

3. Backend & API Reference

Base URL: `/api/v1`. Responses use `{ success, message, data, timestamp }`. Auth: `Authorization: Bearer <accessToken>`; refresh via `/auth/refresh`.

3.1 Layout

`src/app.js` (Express + Socket.IO), `config/` (db, redis, env, secrets), `middlewares/` (auth, validation, rate limits, errors), `modules/<feature>/` (routes → controller → service → model), `sockets/`, `services/`, `utils/`.

3.2 Modules

admin (incl. rbac, audit log, dashboard, settings), ads, ai-booking, analytics, auth, branding, broadcasts, calls (masked calling), chat, corporate, dispatch, disputes, drivers, earnings, loyalty, maps, notifications, payments, pricing (incl. surge), privacy, promos, referrals, revenue, rides, safety (SOS), scheduled-rides, subscriptions, tracking, users, vehicles, wallet, zones.

3.3 Auth

Endpoint	Purpose
`POST /auth/send-otp` `{phone}`	Send OTP (Twilio/MSG91). `DEMO_MODE=true` → OTP is `123456`, SMS skipped
`POST /auth/verify-otp` `{phone, otp, role, firstName?, lastName?}`	Login/signup; **`role` required** (`passenger` / `driver`); returns `{user, accessToken, refreshToken, sessionId, isNewUser}`
`POST /auth/social`, `/auth/register`, `/auth/refresh`, `/auth/logout`, `GET /auth/me`, `GET /auth/sessions`	
`POST /auth/admin-login` `{email, password}`	Admin panel login

3.4 Core endpoints

Rides: `POST /rides/estimate`, `POST /rides`, `GET /rides/active`, `GET /rides/history`, `GET /rides/:id`, `POST /rides/:id/accept` (driver), `POST /rides/:id/verify-pickup-otp`, `PATCH /rides/:id/status`, `GET /rides/:id/cancellation-fee`, `POST /rides/:id/cancel`, `POST /rides/:id/rate`, `POST /rides/:id/share`, `GET /rides/nearby-drivers`, `GET /rides/availability`. **Drivers:** `GET/PATCH /drivers/me`, `PATCH /drivers/me/status`, `PATCH /drivers/me/location` (`{latitude, longitude}`), `GET /drivers/me/earnings`, `/me/rides`, documents upload endpoints, `POST /drivers/me/complete-onboarding`. **Payments/Wallet:** `GET /payments/gateways`, `POST /payments/intent|pay|charge-card|charge-wallet|charge-external`, cards CRUD, `POST /payments/:id/refund` (admin), webhooks (`/payments/webhook`, `/payments/webhooks/:gateway`); `GET /wallet/balance|history`, `POST /wallet/topup`. **Safety:** `POST /safety/sos`, `GET/DELETE /safety/sos/me`. **Users:** `GET/PUT /users/me/saved-addresses`, emergency contacts, profile. **Promos:** `POST /promos/validate`. **Subscriptions:**

`/subscriptions/plans|my|subscribe|cancel`. **Admin (`/admin/*`, admin role):** dashboard, analytics, users, drivers (approve/reject/suspend/…), rides, pricing, promos, documents & document-types, SOS, live rides/drivers, disputes, scheduled rides, admin-roles, zones, vehicle-categories, broadcasts, audit-log, referrals, corporate, payouts (`/payouts`, `/payouts/driver-earnings`, `/payouts/initiate`, approve/reject/retry), surge-zones/events, vehicles, settings.

3.5 Realtime (Socket.IO)

Rooms: `driver:<id>`, `passenger:<id>`, ride rooms, `admin:live`. Events: `driver:online`, `driver:location`, `ride:offer`, `ride:response`, `ride:accepted`, `ride:status`, `ride:otp_verified`, `ride:cancelled`, `ride:join|leave`, `chat:message|read`, `typing`, `call:*`, `sos:triggered|resolved`, `scheduled:*`, `dispatch:unassigned`, `dispute:resolved`, `heartbeat`.

3.6 Data model (Mongo collections)

users, drivers, vehicles, vehicle-categories, rides, payments, wallet transactions, driver earnings, payout requests, driver bank accounts, revenue ledger, pricing, surge zones/events, zones, promos, referrals, subscriptions & plans, notifications (+templates, logs), sos alerts, disputes, chat, call sessions, scheduled rides, documents & document types, admin roles, audit logs, settings/integration settings, corporate (accounts, employees, trips, invoices), loyalty, ads, broadcasts, branding, privacy exports, driver strikes, dispatch logs, AI booking sessions. Key ride fields: `passenger`, `driver`, `status`, `pickup/dropoff {address, location(GeoJSON)}`, `rideType`, `fare{baseFare,distanceFare,timeFare,surge,discount,tax,total,platformFee,driverEarning,currency}`, `paymentMethod/Status`, `timeline{…}`, `pickupOtp`, `passengerRating/driverRating {rating, review, ratedAt}`.

3.7 Cross-cutting

Rate limiting on OTP/social/admin/webhooks; helmet/CORS; Winston logs; audit log on selected admin writes; encrypted integration secrets (`ENCRYPTION_KEY`); health at `/api/v1/health`; background crons (surge sweep 2 min, dashboard KPI emit 30 s, SOS escalation, calls cleanup).

4. Mobile Apps (Expo / React Native)

4.1 Structure

`App.tsx`, `app.config.js` (variant → bundle IDs `com.gentaxi.passenger` / `com.gentaxi.driver`), `src/navigation/RootNavigator.tsx`, `src/screens/{auth,onboarding,passenger,driver,ride,payment,settings,shared,sos,chat,notifications}`, `src/store` (Zustand stores + Redux slices), `src/services/api/*` (axios client with token refresh), `socket.service.ts`, `src/i18n` (custom engine: `{t, locale, isRTL}`; FR/AR/EN), `src/theme`.

4.2 Configuration (`.env`, `EXPO_PUBLIC_*`)

`API_URL`, `GOOGLE_MAPS_KEY`, `STRIPE_PUBLISHABLE_KEY`, `ENABLE_AI_BOOKING`, `AI_GATEWAY_URL`, `CHAT_ORCHESTRATOR_URL`, `INSIGHTS_URL`, optional `DEFAULT_LAT/LNG` (fallback map centre), `MOCK_NOTIFICATIONS=1` (dev-only fake notifications).

4.3 Run in development

```
cd gen-taxi-frontend
EXPO_PUBLIC_API_URL=http://localhost:5001/api/v1 APP_VARIANT=passenger npx expo start --dev-client --port 8082
EXPO_PUBLIC_API_URL=http://localhost:5001/api/v1 APP_VARIANT=driver    npx expo start --dev-client --port 8081
```

Use the installed dev clients (iOS simulator / Android) and pick the Metro URL.

4.4 Build & release

- Android APK (local EAS, ~2–5 min each): `APP_VARIANT=driver EAS_NO_VCS=1 npx eas-cli build --local -p android --profile preview-driver --output ./gentaxi-driver-latest.apk --non-interactive` (and `passenger` / `preview-passenger`). Needs Java 17+, `ANDROID_HOME`.
- OTA updates: publish to branches **`preview-driver`** and **`preview-passenger`** (not `main`); double-restart the app to apply.
- Hermes limitation: no `AbortController` or regex `s` flag — avoid them in client code.
- Screenshots/automation: Maestro flows in `.maestro/`.

4.5 Notable behaviours

Pickup defaults to current GPS; dev builds may read `Documents/dev_user.json` to bypass login (driver, `__DEV__` only); SOS needs a GPS fix (falls back to last-known); notification screen uses the real API unless mock flag set.

5. Admin Panel

React + Vite + Tailwind. Env: `VITE_API_URL`, `VITE_SOCKET_URL`, `VITE_GOOGLE_MAPS_API_KEY`, `VITE_MAP_DEFAULT_LAT/LNG`, AI service URLs (`VITE_AI_GATEWAY_URL`, `VITE_CURRENCY_SERVICE_URL`, `VITE_CHAT_ORCHESTRATOR_URL`, `VITE_INSIGHTS_SERVICE_URL`, `VITE_DEMAND_SERVICE_URL`). Run: `npm run dev` (port 5173). Seed an admin with `node scripts/seedAdmin.js` in the backend (**change the default password immediately**).

Route	What it manages
`/dashboard`	KPIs (active rides, online drivers, revenue today, SOS, pending approvals), charts
`/analytics`	Overview, driver league, retention, exportable reports (CSV/PDF)
`/live-map`	Active rides, drivers, SOS alerts with triage/resolve
`/dispatch`	Manual (phone/walk-in) bookings, online drivers
`/rides`	All rides, filters, ride details, live monitor, scheduled
`/users`, `/drivers`	Lists, detail drawers, block/suspend/ban, driver approval, documents, audit trail
`/vehicles`	Fleet CRUD, verify, assign to driver
`/payouts`	Driver earnings, payout requests, transaction history, pay
`/pricing`, `/pricing/surge-zones`	Per-ride-type fares and commission; geofenced surge zones, live monitor, events
`/promos`	Promo codes and referral programme
`/subscriptions`	Driver plans and subscribers
`/ride-types`, `/document-types`	Categories; required driver documents
`/settings`	Dispatch radius; Maps, Payments, SMS, Email, Push, Call masking, Storage, Analytics integrations (secrets encrypted)
`/audit-log`	Append-only admin action log
`/platform`, `/branding`	Marketing/branding pages
`/ai/*`	cost, currencies, knowledge base, handoff inbox, review insights, surge, AI ops

RBAC: admin roles are managed via `/admin/admin-roles` (no dedicated UI page).

6. AI Platform

Separate pnpm workspace at the repo root (see also the phase docs `AI_PLATFORM_PHASE1..5.md`, `AI_DEPLOYMENT.md`, `AI_LOCALIZATION_QA.md`).

Service	Port	Purpose
`ai-gateway` (NestJS)	8080	Single entry to LLMs: JWT+RBAC, throttling, token budget, PII redaction, Groq + Anthropic router with circuit breaker/fallback, prompt versioning, RAG store (Atlas `$vectorSearch` with in-memory cosine fallback), call logs, cost API, demand proxy
`currency-service`	8081	FX rates, Money value object
`chat-orchestrator`	8082	Support chatbot: tool-calling loop (max 5 hops), 7 tools with server-side authz calling the Backend REST API with the caller's JWT, RAG, grounding check, confirmation-gated book/cancel, refund → admin queue, SSE streaming, escalation + admin handoff inbox
`insights-service`	8083	Batch review pipeline: aspect extraction, per-driver summaries, zone aggregation; guardrails (cold start <5, toxicity/PII, review-bombing)
`demand-service` (FastAPI)	8084	H3 demand actuals, seasonal baseline + LightGBM, advisory surge engine, heatmap
`packages/ai-shared`	—	Shared types/interfaces
`ai-eval`	—	Golden-set regression gate (wired into CI)

Models and config

Chat models default to Groq `openai/gpt-oss-120b` (chat) and `openai/gpt-oss-20b` (cheap/aspects) — the earlier llama-3.x models were decommissioned. `GROQ_BASE_URL` must be host-only (`https://api.groq.com`). Embeddings: `EMBEDDING_PROVIDER=local` (hash-based 1024-dim; Cohere is the intended production provider but no key is configured).

Production facts

- Deployed on Render (Singapore, Docker, same Atlas DB and Redis as the backend). Auto-deploy does **not** fire on push: trigger via `POST /v1/services/{id}/deploys`.
- Each service must declare every dependency it imports (workspace hoisting hides gaps locally; Render clean builds expose them).
- Atlas Vector index `kb_chunks_vector` must be created (`ai-gateway/scripts/create-atlas-index.sh`); until then search uses the cosine fallback.
- The production knowledge base was empty; draft policy docs are in `ai-gateway/kb-drafts/` awaiting review (`[CONFIRM]` markers).
- Demand/insights output needs real ride history (empty in a fresh environment).

7. Setup, Deployment & Operations

7.1 Local setup (backend)

Prereqs: Node 20, MongoDB 7, Redis 7.

```
cd gen-taxi-backend && npm install
cp .env.example .env     # set MONGO_URI, JWT_SECRET, JWT_REFRESH_SECRET, ENCRYPTION_KEY …
export MONGO_URI=mongodb://localhost:27017/genxtaxi_dev PORT=5001 DEMO_MODE=true
node scripts/seedAdmin.js && node scripts/seedDocumentTypes.js
node src/app.js           # health: GET /api/v1/health
```

Admin: `cd gen-taxi-admin && npm install && VITE_API_URL=http://localhost:5001/api/v1 npm run dev`. Mobile: see doc 4. AI services: `pnpm install` then `pnpm dev:gateway` etc., or `docker compose -f docker/docker-compose.ai.yml up`.

7.2 Backend environment variables (names only)

Core: `NODE_ENV PORT MONGO_URI REDIS_HOST/PORT/PASSWORD JWT_SECRET JWT_REFRESH_SECRET ENCRYPTION_KEY DEMO_MODE`. Auth: `GOOGLE_OAUTH_CLIENT_ID(S) APPLE_CLIENT_ID(S)`. Payments: `PAYMENT_PROVIDER STRIPE_* RAZORPAY_*`. Messaging: `SMS_PROVIDER TWILIO_* MSG91_* EMAIL_PROVIDER SENDGRID_API_KEY SMTP_* PUSH_PROVIDER FIREBASE_*`. Maps/storage: `GOOGLE_MAPS_API_KEY AWS_*`. Pricing defaults: `BASE_FARE PER_KM_RATE PER_MIN_RATE DRIVER_COMMISSION_RATE PLATFORM_FEE_RATE SURGE_*`. AI services additionally need the **same** `JWT_SECRET`, `MONGODB_URI`, `REDIS_URL`, `GROQ_API_KEY`, `ANTHROPIC_API_KEY`, `INSIGHTS_SERVICE_TOKEN`, `DEMAND_SERVICE_URL`.

7.3 Deployment (Render)

Services: `gentaxi-backend` plus the five AI services (`render.yaml` blueprint, Docker). Redeploy API: `POST https://api.render.com/v1/services/{id}/deploys`. Order: backend first when schema/behaviour changes, then AI services. Free plans sleep — warm them before demos. Release checklist: run tests → merge → deploy backend → smoke test (`/health`, login, estimate) → publish OTA / build APKs.

7.4 Operations runbook

- **Chatbot says "briefly unavailable":** check gateway model env vars and Groq model availability; look at circuit breaker in gateway logs.
- **Rides stuck in `searching`:** no online driver within radius for the tier; check `drivers:online:*` in Redis and driver `currentLocation`.
- **AI features slow after idle:** Render cold start; warm `/health`.
- **Wallet top-up 503:** payment gateway keys not configured.
- **Payout "No earnings":** period/ledger mismatch; verify `DriverEarning` rows and already-paid totals.

7.5 Security notes

Rotate any credential that has appeared in chat/logs (Render API key, admin password). Do not commit `.env` or the Firebase admin JSON (`genxtaxi-firebase-adminsdk-*.json` sits in the repo root — move it out and rotate if it was ever pushed). Change the seeded admin password. Keep `DEMO_MODE` **off** in production (OTP bypass). Integration secrets are stored AES-encrypted via `ENCRYPTION_KEY` .

8. Testing & Known Issues

8.1 Automated

Backend: Jest (`npm test`). AI platform: 43 Node + 15 Python tests plus the `ai-eval` golden-set gate. Demand service: `pytest` (surge engine, baseline, bbox).

8.2 End-to-end run (Oct 2026)

A full local run drove the rider app, driver app and admin panel (iOS simulators + Chrome) with US test data (rider Daniel Carter, driver Michael Anderson, NYC routes, 30+ rides). Screenshots and the per-feature coverage table are in `screenshots/e2e/` (`REPORT.md`). Verified end to end: signup, booking with promo, dispatch/accept, OTP, trip lifecycle, fare/earnings, ratings, history/receipts, cancel with fee, SOS → admin triage, surge zone → rider banner, payouts, admin dashboards/lists/settings, audit log.

8.3 Fixed during testing (committed Oct 2026)

Audit log page empty; USD/INR default mismatch; stale/hardcoded profile stats; rating stars; map fallback city; duplicate SOS and 0,0 location; payout period timezone bug; fake dev notifications; OTP keyboard hiding Verify; demand heatmap 422; vector-search empty-result fallback.

8.4 Open issues / not done

- Mobile screens hard-code `₹` (no real multi-currency).
- Wallet top-up failure shows no error to the user; SOS confirmation claims contacts were notified even with none.
- Gateway returns HTTP 200 for upstream 422 on demand proxy.
- Driver "Online hrs" does not accumulate.
- Atlas vector index and production knowledge base still to be created/published.
- Admin has no UI for roles, broadcasts, zones, loyalty, corporate.
- ride-offer window (~3 s modal on some flows) is easy to miss; consider longer countdown.

8.5 Not tested

Splash/onboarding, driver registration with document upload, driver decline/cancel, in-app card payment screen, sign-out, chat/calls, push notifications, AI assistant/My Feedback/admin AI pages against a live AI stack, real payment gateways.